Easy Chain: project report, all phases

As of 2026-10-23. Version 0.4.0, commit `d6d73be` on `main`. CI is green on all five jobs: Linux, Windows, Playwright, Docker and Compose, and the web build.

This report covers the whole build in one place. Phases 0 to 4 are done; Phases 5 and 6 are planned. Each phase also has its own detailed report, linked in its section. The live status is in handover/status.md.

Summary

What Easy Chain is. An open-source, drag-and-drop builder for LLM apps and AI agents, for people who don't write code every day. You draw a flow on a canvas, press Run and watch it work. Every flow compiles to **plain LangGraph Python**: that same code runs when you press Run, and it is what you get when you export. An exported flow needs no Easy Chain to run.

Where it stands. Five of the seven phases are done, and each met its "Done when". Work is paused for your review of Phase 4, as the build rules ask, and Phase 5 starts when you answer the questions at the end of this report.

Phase	What it delivered	State
0. Foundations	The flow format, the compiler, the runtime, the command line, one-command start	✅ Done
1. Visual MVP	The canvas editor, running and debugging, export	✅ Done
2. Real runtime	Durable runs that survive crashes, people in the loop, loops and parallel work, triggers	✅ Done
3. Agents and knowledge	Agents with tools, Knowledge Bases with citations, 14 model providers	✅ Done
(Review)	A full code review worked through, and Windows support	✅ Done
4. Autopilot and teams	Deep Agents and Helpers, Skills, agent teams, a Code sandbox, the describe-it copilot	✅ Done
5. Platform	Logins and roles, evaluation, publishing, environments, dashboards	Next, after your review
6. Ecosystem	JavaScript export, a step registry, collaboration, Kubernetes	Planned

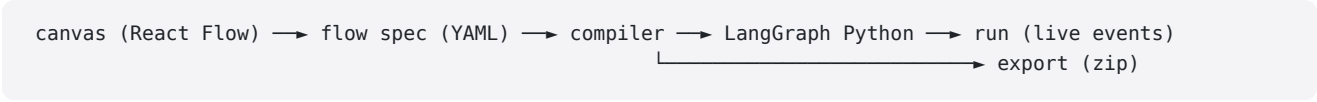
What works today.

- **18 step types**, from AI Model and Decision to Agent, Deep Agent, Knowledge Base search and Ask a Human, each with a docs page.
- **13 templates**, each passing a Test Set of 10 to 12 cases.
- **14 model providers**, plus a key-free **stand-in AI** so anyone can try a flow before adding a key.
- **Durable runs:** runs survive crashes, wait for people as long as needed in the **Inbox**, and can be rewound to any **Save Point**.

- **Tests on Linux and Windows:** 513 Python tests, 94 web unit tests, 11 client tests and 39 browser journeys run on every push, with accessibility checks.

What it needs from you. Five decisions about Phase 5 (see Questions for you), and, before any release, a check of the agent templates with a real model key.

How it works



1. A flow is a readable YAML file (the **flow spec**), kept in a folder you can put in git.
2. The **compiler** checks it and turns it into a LangGraph module:
   - Flow Data becomes the graph's state;
   - steps become nodes;
   - Decisions become conditional edges.
3. **Run** puts the run on a queue. A **worker** runs exactly that module, saving a Save Point after each step, and the canvas follows its events live. **Export** writes the same module to a zip.

Easy Chain does not reimplement an agent or graph engine; it uses LangGraph, LangChain and deepagents, pinned to exact versions. The design and every deliberate difference from the build prompt are in ARCHITECTURE.md.

Phases 0 and 1: foundations and the visual editor

Detailed report: phases/phase-0-1.md.

Done when	Evidence
One command starts the app	`docker compose up` (health check, web app, a chat run inside the container), or `pnpm dev`
A 3-step YAML flow compiles to LangGraph and runs from the command line	`easychain run examples/hello.flow.yaml … --stand-in`, with a test
A new user builds and runs a summarise-this-URL flow in under 5 minutes, without docs	A browser test builds it from a blank canvas and runs it in about 6 seconds (5-minute ceiling). It is scripted; a test with real users is still recommended.
The exported Python runs unchanged	Tests download the export and run it with plain Python and the real OpenAI client, with Easy Chain hidden

What was built:

- **Foundations:**
  - a monorepo (Python and the web app) with CI and Docker;
  - flow spec v1 with a published JSON Schema;
  - the compiler, with golden files that pin the generated code;
  - the runtime;

    - the `easychain` command line.
- **The editor:**
    - drag and drop from a searchable step library;
    - connections checked as you draw them, with the reason shown when one is refused;
    - Decisions with labelled exits;
    - sticky notes, auto-layout, undo and redo, copy and paste, keyboard shortcuts.
- **Steps:** Input, Output, AI Model, Instructions, Web request, Code and Decision (rules, safe expressions, or an AI picking the exit).
- **Running:**
    - a form or a streaming chat;
    - the active step glows, tokens stream inside it, and each step shows its time, tokens and cost;
    - a trace of what each step read and saved;
    - plain-language errors with one-click fixes;
    - Run Replay plays a past run back on the canvas.
- **Export:** a zip with the module, `requirements.txt`, `langgraph.json`, `.env.example` and a README.
- **Modes:** Beginner and Pro, with LangChain terms shown only in Pro, and light and dark themes.
- **Templates:** the first four.

At the end:

- **Tests:** 170 Python tests, 14 web unit tests and 11 browser journeys.
- **Performance:** about 1 ms of overhead per step (the budget is 10 ms). A 300-step flow checks in about 40 ms and opens on the canvas in about 0.7 s.
- **Accessibility:** no serious WCAG 2.2 AA issues in either theme.

Phase 2: a real runtime

Detailed report: phases/phase-2.md.

Done when	Evidence
Killing a worker mid-run and restarting resumes from the last step	Real worker processes are killed with SIGKILL half-way through a step. A new worker finishes the run without repeating finished steps. Tested on SQLite and Postgres, 18 runs in a row.
…with no duplicate side effects	A payment request interrupted by a crash is sent again with the same Idempotency-Key, so it happens once. Retries do the same.
A paused approval can be resumed a day later	The run pauses, everything stops, and the clock is moved back 24 hours. A fresh server and worker then take the answer from the Inbox and finish the run.

What was built:

- **Graph features:**

    - the **Flow Data panel**, with update rules: replace, add to the list, add up, merge, or custom;
    - loops with round limits, and parallel branches that wait for each other;
    - **For Each**, with a concurrency limit;
    - **Sub-flows** and **Jump**;
    - **Ask a Human**: approve, edit, answer or choose;
    - per-step retries, time limits and caching.
- **Side effects run at most once.** POST requests and Code marked *run at most once* record their result and send an `Idempotency-Key`.
- **Durability:**
    - Save Points in SQLite or Postgres;
    - a run database, and a job queue with leases and takeover of a dead worker's jobs;
    - `easychain worker` scales out.
- **Debugging:**
    - breakpoints before or after any step;
    - **time travel**: run again from any Save Point, with edited Flow Data;
    - stop a run and carry it on later.
- **Following a run:** live events over SSE and WebSocket, so a run survives a page reload.
- **The Inbox,** with notifications by webhook, Slack and email.
- **Triggers:** webhook, schedule (cron with time zones), file upload, email, and after another flow. Chat flows have rules for a second message arriving while the first is still running.
- **Packaging:** Docker Compose with Postgres and workers, and `@easychain/client`, a TypeScript client with a React hook.
- **Templates:** Approval workflow and Summarise several pages.

At the end: 259 Python tests, 19 web unit tests, 4 client tests and 23 browser journeys; compiler coverage 94%.

Phase 3: agents and knowledge

Detailed report: phases/phase-3.md.

Done when	Evidence
The **Support bot over docs** template passes its Test Set	12/12. It answers from a sample help centre with the right citation, keeps the chat, and hands off-topic questions to a person.
The **SQL analyst** template passes its Test Set	10/10. It answers from a sample shop database. Read-only blocks a `DELETE`, emailing a report waits for approval, and the tool-call limit stops a runaway agent.

What was built:

- **The Agent step** (`create_agent`):
    - other steps become its tools when you drag them onto its Tools handle;
    - tool calls show live on the canvas;

    - **Add-ons** behind switches: ask a person before a tool runs, call limits, retries, fallback models, summarising long chats, personal-data filters, a to-do list, long-term memory.
- **Structured replies.** A field builder for the AI Model and the Agent. Replies are checked, retried when they don't fit, and can go straight into Flow Data.
- **Knowledge Bases:**
    - add PDF, Word, HTML, Markdown, CSV, text or web pages, and preview how they split;
    - search by meaning, by words, or both;
    - answers cite `[1]`, linked to the passage's page and heading;
    - stored in pgvector on Postgres, or locally on SQLite.
- **New steps:** Memory (remember facts about a user, keep long chats short), Database query (read-only unless you say otherwise), and MCP tool.
- **Connections out: MCP servers** in Settings, and **Import an API** from an OpenAPI description.
- **14 model providers:** OpenAI, Anthropic, Ollama, Google Gemini and Vertex AI, AWS Bedrock, Azure OpenAI, Mistral, Groq, Together, Fireworks, OpenRouter, DeepSeek and xAI.

At the end: 320 Python tests, 32 web unit tests, 4 client tests and 31 browser journeys.

Between Phases 3 and 4: the code review and Windows

Details: handover/review.md.

A full read-through of the code found it well built, but with some serious issues:

- bugs that could lose data silently;
- gaps in the security of the local setup;
- running from source on Windows was broken.

Thirteen "fix first" findings were fixed, each with a test. Among them:

- a step name with a line break could become code;
- local MCP servers received every saved secret;
- a wrong secret key could wipe the vault;
- an Inbox answer could go to the wrong pause;
- the server had no Host or Origin check;
- autosave lost the last edits when you left a flow.

Easy Chain now runs on Windows as well as Linux and macOS (your decision R.4), and CI tests Windows on every push. Two review items were left on purpose:

- **an egress allow-list** for fetches made by the server, which comes with Phase 5's roles;
- **a real-model check** of the agent templates, before each release.

Phase 4: autopilot and teams

Detailed report: phases/phase-4.md.

Done when	Evidence
A **Research assistant** template (Deep Agent + Helpers + to-do list + files) passes a 10-case Test Set	10/10, including one case where nothing is scripted. Its exported code also runs on its own, with its skills inside.
Code steps can't escape their sandbox	Tests with real containers try to read the host's files, reach the network, read secrets, become root, reach Docker itself, write outside `/tmp`, fork-bomb, and exceed memory and time. Every attempt fails.

What was built:

- **Code sandbox.** Each Code step call runs in a fresh, locked-down Docker container, using gVisor when it is installed. **Settings → Sandbox** shows whether Docker was found and sets the limits.
- **Deep Agent** (deepagents' `create_deep_agent`):
  - for longer tasks: it plans with a to-do list, keeps notes and drafts in files, and hands sub-tasks to Helpers;
  - the files are kept with every Save Point, so they survive a crash and rewind with time travel.
- **Helpers:**
  - specialists under a Deep Agent, each with its own role, model and tools;
  - their work nests under the task that started it in the trace, with its own tokens and cost;
  - their approvals go to the Inbox.
- **Skills:**
  - a Skills library in the Agent Skills format (`SKILL.md` plus files);
  - agents read a skill when it fits the task;
  - exports carry the skills.
- **Agent teams.** Supervisor, handoffs and plan-and-execute, made of ordinary steps, so you can change them and they export to plain LangGraph. Insert one from the step library.
- **The describe-it copilot:**
  - describe a flow or a change in words;
  - the proposal is checked before you see it, previewed on the canvas, and you keep the changes you want, as one undo step;
  - it uses any model you have a key for.
- **Templates:**
  - **Research assistant**: a Deep Agent with a researcher and a reviewer;
  - **Inbox triage**: a supervisor, where urgent email waits for a person;
  - the three agent-team templates.

At the end: 513 Python tests (none skipped), 94 web unit tests, 11 client tests and 39 browser journeys; compiler coverage 94%; all 13 template Test Sets pass.

Phase 5 plan: Platform

This phase turns a single-user tool into a multi-user server. The full plan is in ROADMAP.md.

Proposed "Done when":

1. A flow moves **draft → staging → production** only when its Test Run passes in CI.
2. A published flow serves an API endpoint and a chat widget, with per-key rate limits.
3. Roles stop a Viewer from editing and an Editor from publishing to production.
4. Every change and publish is in the audit log.

#	Milestone	What it delivers
5.8	Platform groundwork	Database migrations (Alembic). Knowledge Base ingestion moves onto the job queue.
5.6	Users, roles, SSO, audit	Logins, roles (Owner, Editor, Runner, Viewer), single sign-on, per-workspace vaults, an audit log, and the egress allow-list.
5.1	Test Sets editor and Checks	Edit cases in the app. Checks include exact values, JSON schema, tool calls, route, cost and time budgets, an AI judge with a rubric, and the order of tool calls.
5.2	Test Runs and baselines	Compare a flow version against a baseline, case by case. Repeat runs to measure flakiness.
5.3	CI gate	`easychain eval` fails CI on a regression; a GitHub Action is included.
5.4	Environments and Publish	Dev, staging and prod environments. Publish gives an API endpoint, a chat page and a widget. `easychain deploy`.
5.5	Model gateway	Budgets and alerts, caching, provider fallbacks, and cost per flow, run and user.
5.7	Dashboards and tracing	Runs, errors, latency and cost over time; OpenTelemetry and LangSmith.

The milestones are listed in the order they will be built (5.8 first). Each milestone includes its own security review.

Phase 6 plan: Ecosystem

Proposed "Done when":

1. The **LangGraph.js** export passes the same Test Set as the Python export, for every template.
2. A custom step published to the registry installs into another workspace and runs.
3. Two people edit the same flow at once without losing changes.
4. The Helm chart deploys to a local Kubernetes cluster and passes the smoke test.

Milestones:

- LangGraph.js export.
- A custom step registry.
- Import from LangGraph code and other builders.

- Real-time collaboration.
- Prompt optimisation from Test Sets.
- A Helm chart.

Quality over time

End of	Python tests	Web unit	Client	Browser journeys	Template Test Sets
Phases 0–1	170	14	–	11	4 of 4 pass
Phase 2	259	19	4	23	6 of 6
Phase 3	320	32	4	31	8 of 8
Phase 4	513	94	11	39	13 of 13

- **CI** runs five jobs on every push:
  - Python on Linux, with Postgres and pgvector;
  - Windows: the Python suite, the command line and a worker, and the web checks;
  - the web build and unit tests;
  - the browser journeys against the real app;
  - Docker and Compose, including a check that Code steps really run in containers.
- **Code checks:** compiler coverage must stay at 90% or more (it is 94%), and golden files pin the generated code.
- **No paid APIs in tests:**
  - tests use the stand-in AI, or a small fake OpenAI-compatible server;
  - agent Test Sets script the model's turns;
  - tools, limits, approvals and routes stay real.
- **Accessibility:** every browser journey runs axe WCAG 2.2 AA scans; no serious issues.
- **Red runs on the way:** two CI runs failed during Phase 4 and were fixed in the next commit:
  - a browser test that matched two status messages;
  - a timing test on the slow Windows runner.

  The lessons are in handover/lessons.md.

Decisions

Every decision is in handover/decisions.md, with who made it and why. The main ones:

#	Decision
2.1–2.3	Flows stay YAML files; runs and Save Points go in SQLite or Postgres. The job queue is a database table, with no Redis. Code ran unsandboxed until Phase 4.
2.4	Apache 2.0 licence.

#	Decision
3.1–3.3	pgvector for Knowledge Bases. Default embeddings: OpenAI, then Ollama, then a key-free option. Local MCP servers only in Pro mode, from an approved list.
R.1–R.3	No AI attribution in git. Work happens on `main`; the old branch was deleted.
R.4–R.5	Windows and Linux both supported. The code review was fixed before Phase 4.
4.1–4.3	Phase 4's "Done when". Docker sandbox, with gVisor where installed. The copilot uses any model with a key.
4.4–4.10	Deep Agent files live in the run's state; there is no shell; skills ship inside exports; agent teams are ordinary steps; the copilot writes the whole flow and it is checked; version 0.4.0.

Known gaps and risks now

- **Security on a shared server:** there is no login yet, and no egress allow-list for fetches the server makes. Keep the server on localhost or behind your own access control until Phase 5.
- **The sandbox needs Docker.** Without Docker, Code steps run in Easy Chain's own process, with a warning. In Compose, the sandbox needs the opt-in overlay, which gives the worker the Docker socket. Exported code runs Code steps in its own process.
- **Real models are not run in CI,** by design. Agent behaviour with real models must be checked by hand before a release.
- **The copilot's quality depends on the model.** It sends the whole flow each time, so very large flows make long prompts.
- **Dependencies:** `deepagents` moves quickly. It is pinned exactly, and golden files catch changes.
- **Platform debt** (all in Phase 5):
  - database migrations are not set up yet;
  - Knowledge Base ingestion runs in the API server;
  - live events poll the database.
- **Smaller gaps** (backlog):
  - no OCR for scanned PDFs;
  - one vector store;
  - MCP supports tools only;
  - OpenAPI import doesn't send optional query parameters.

Questions for you

These decide how Phase 5 is built. My recommendation is in *italics*.

1. **Phase 5 "Done when":** are the four criteria above right?
2. **Single sign-on:** which provider should the final check use: Okta, Google, Microsoft Entra, or none for now? *None for now, with setup documented for all three.*

3. **Publishing:** self-hosted only, or also a hosted cloud? *Self-hosted only in Phase 5.*
4. **The AI judge in Checks:** which model grades answers? *Any model you have a key for; with no key, those checks are skipped with a note.*
5. **Real-model checks before a release:** by you, or by a manual CI job with a budgeted key? *A manual CI job, so the result is recorded.*

Reply "go on" to accept all five recommendations, or change any of them.

Where to find things

What	Where
Live status and how to pick the project up	handover/status.md, AGENTS.md
Each phase's full report	phase-0-1, phase-2, phase-3, phase-4
The plan for what's left	ROADMAP.md
How it is designed	ARCHITECTURE.md
Every document	docs/README.md
Running it	README.md: `docker compose up`, or `pnpm run setup && pnpm dev`